PIMPRI CHINCHWAD EDUCATION TRUST's.

PIMPRI CHINCHWAD COLLEGE OF ENGINEERING

(An Autonomous Institute)

B. TECH Year: 2026 – 27 **Semester:** VII **Name of the Student:** Rutuja Jagtap **PRN:** 123B1B146 **Department:** Computer Engineering **Division:** C-1 **Course:** DevOps **Course Code:** BCE27PE01

Assignment No – 3

1. Title

Study and Implementation of Infrastructure as Code (IaC) Using Terraform

2. Aim

To study the concepts and workflow of Infrastructure as Code (IaC), install and configure Terraform, and develop a Terraform configuration to provision an AWS EC2 instance using provider, resource, variable, and output configurations.

3. Objectives

· To understand the concept of Infrastructure as Code.

· To understand the purpose and advantages of Terraform.

· To install and configure Terraform.

· To understand Terraform configuration files.

· To configure an AWS provider using Terraform.

· To create an AWS EC2 instance using Terraform.

· To use variables for configurable infrastructure parameters.

· To use outputs to display important infrastructure information.

· To understand the Terraform workflow such as init, plan, apply, and destroy.

· To understand how infrastructure can be created and managed automatically using code.

4. Introduction to Infrastructure as Code

Infrastructure as Code (IaC) is a method of managing and provisioning computing infrastructure using machine-readable configuration files instead of manually configuring resources through a graphical user interface.

In a traditional approach, an administrator manually opens the cloud console, creates a virtual machine, configures networking and storage, selects the required settings, and starts the server. With IaC, these configurations can be described in code and executed repeatedly.

· Traditional approach: infrastructure is configured manually through a cloud console.

· IaC approach: infrastructure is declared in configuration files and managed through automation.

5. Terraform

Terraform is an Infrastructure as Code tool used to define and provision infrastructure using configuration files. Terraform uses providers to communicate with cloud platforms and other services. In this practical, the AWS provider is used to provision an EC2 instance.

6. Why Terraform?

6.1 Automation
Infrastructure can be created automatically by executing Terraform commands.

6.2 Repeatability
The same configuration can be used to create similar infrastructure multiple times.

6.3 Version Control
Terraform configuration files can be stored in Git or another version-control system.

6.4 Consistency
Infrastructure is created according to the defined configuration rather than depending on manual steps.

6.5 Multi-Cloud Support
Terraform supports many providers, allowing infrastructure to be managed across different platforms.

6.6 Easy Destruction
Resources managed by Terraform can be removed using terraform destroy.

7. Terraform Workflow

The practical follows the standard Terraform workflow: write configuration → initialize → validate → format → plan → apply → verify → destroy.

1. Write Terraform configuration files.
2. Run terraform init to initialize the working directory and download providers.
3. Run terraform validate to check the configuration.
4. Run terraform fmt to format the configuration.
5. Run terraform plan to preview the changes.
6. Run terraform apply to provision the infrastructure.
7. Verify the created resource in AWS and inspect Terraform state.
8. Run terraform destroy after the practical to remove the managed resource.

8. Important Terraform Concepts

8.1 Provider
A provider allows Terraform to communicate with an external platform. The AWS provider is used in this assignment.

8.2 Resource
A resource represents an infrastructure component that Terraform creates and manages.
aws_instance represents an EC2 instance.

8.3 Variable
Variables allow values to be passed into Terraform configurations instead of hard-coding them.

8.4 Output
Outputs display useful information after Terraform creates infrastructure, such as instance ID, public IP and public DNS.

8.5 State

Terraform maintains a state file containing information about infrastructure managed by Terraform. The default file is terraform.tfstate.

9. Prerequisites

Requirement	Purpose
AWS Account / Access	Cloud infrastructure
AWS Credentials	Authentication
Terraform	Infrastructure as Code tool
Terminal / PowerShell	Execute Terraform commands
Internet Connection	Communicate with AWS and download the provider
Text Editor / VS Code	Create .tf configuration files

10. Installing Terraform

Terraform was installed on Windows and verified from PowerShell using:

terraform -version

The command verifies that Terraform is available in the system PATH. The installed version may vary depending on the version available at the time of installation.



11. Configure AWS Credentials

Terraform requires authenticated AWS access to communicate with AWS. Credentials should be configured through a supported AWS authentication mechanism. Credentials must never be placed directly inside Terraform source files or submitted in screenshots.

aws configure

· AWS Access Key ID

· AWS Secret Access Key

· Default region: ap-south-1 (Mumbai)

· Output format

12. Create Terraform Project

The project directory used for the practical is:

```
terraform-aws-ec2/
├── main.tf
├── variables.tf
├── outputs.tf
├── .terraform.lock.hcl
└── terraform.tfstate
```

The implementation shown in the supplied screenshots uses main.tf, variables.tf and outputs.tf. Terraform also creates supporting files such as .terraform.lock.hcl and terraform.tfstate during execution.

13. Provider and Resource Configuration

The implementation uses the AWS provider version compatible with the 6.x provider series and the Mumbai region (ap-south-1). The AMI is obtained dynamically from AWS Systems Manager Parameter Store rather than hard-coding a region-specific AMI ID.

```
terraform {
 required_providers {
 aws = {  source =
    "hashicorp/aws"
    version = "~> 6.0"
   }
 }} provider
"aws" {
 region = "ap-south-1"
}
data "aws_ssm_parameter" "amazon_linux_2023" {
 name =
"/aws/service/ami-amazon-linux-latest/al2023-ami-kernel-default-x86_64" }  resource
"aws_instance" "web_server" {
 ami = data.aws_ssm_parameter.amazon_linux_2023.value
 instance_type = var.instance_type

 tags = {
  Name = var.instance_name
 }
}
```

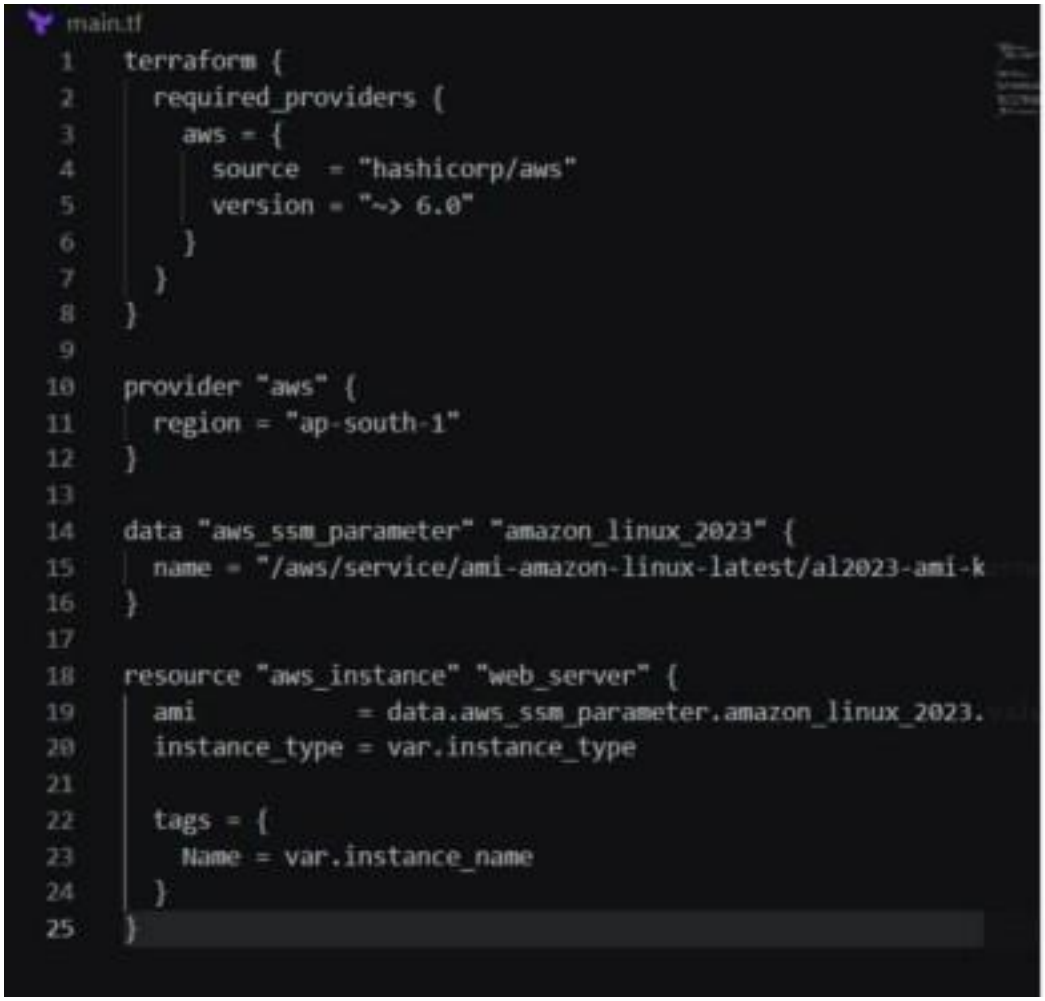


Figure 1: main.tf – AWS provider, AMI data source and EC2 resource

14. Variable Configuration

The variables used by the EC2 resource are declared in variables.tf:

```
variable "instance_type" {
 description = "EC2 instance type"
 type = string
 default = "t3.micro"
}

variable "instance_name" {
 description = "Name of the EC2 instance"
 type = string
 default = "Terraform-EC2"
}
```

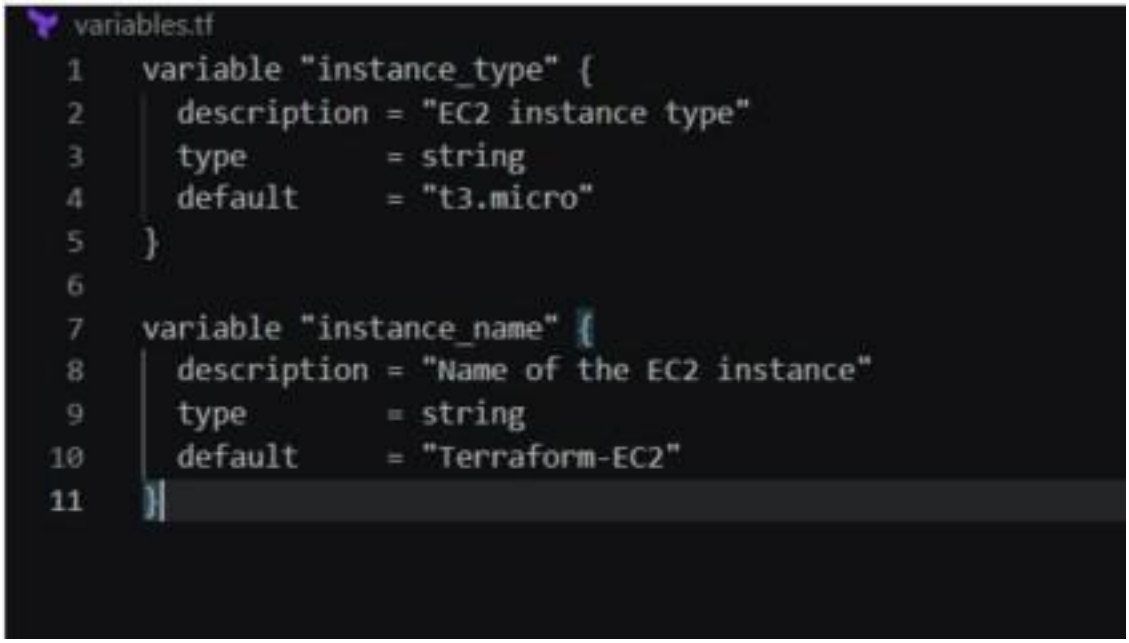


Figure 2: variables.tf – instance type and instance name variables

15. Output Configuration

The outputs.tf file exposes useful information about the EC2 instance:

```
output "instance_id" {
  description = "ID of the created EC2 instance"    value
  = aws_instance.web_server.id
} output "instance_public_ip"
{
  description = "Public IP address of the EC2 instance"    value
  = aws_instance.web_server.public_ip
}

output "instance_public_dns" {
  description = "Public DNS name of the EC2 instance"    value
  = aws_instance.web_server.public_dns
}
```

outputs.tf

```
output "instance_id" {
  description = "ID of the created EC2 instance"
  value       = aws_instance.web_server.id
}

output "instance_public_ip" {
  description = "Public IP address of the EC2 instance"
  value       = aws_instance.web_server.public_ip
}

output "instance_public_dns" {
  description = "Public DNS name of the EC2 instance"
  value       = aws_instance.web_server.public_dns
}
```

Figure 3: outputs.tf – Terraform output definitions

16. Terraform Initialization

The working directory was initialized using:

terraform init

Terraform downloaded and installed the required AWS provider. The supplied screenshot shows successful initialization and installation of the hashicorp/aws provider.

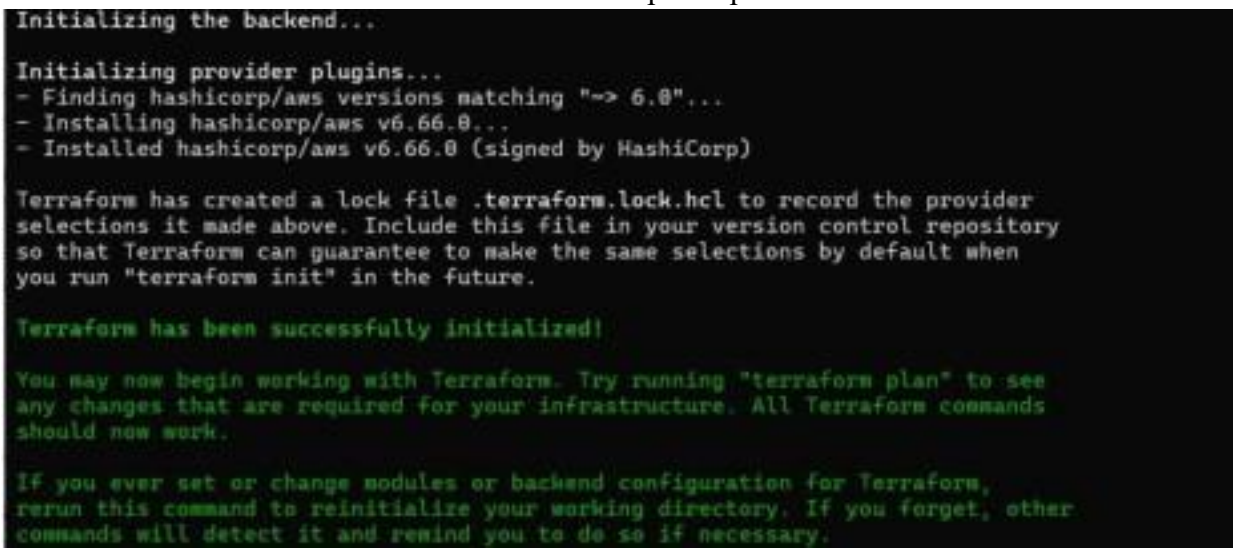


Figure 4: terraform init – successful initialization

17. Validate Configuration

The Terraform configuration was checked using:

terraform validate

The supplied execution result reports: Success! The configuration is valid.

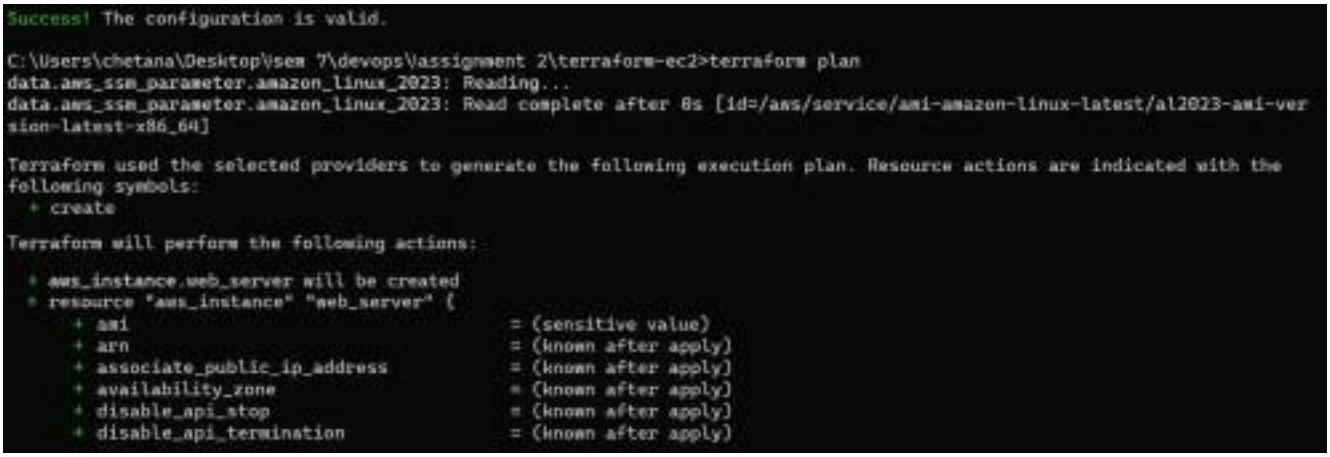


Figure 5: terraform validate – configuration is valid

18. Format Terraform Files

Terraform configuration files can be formatted using:

terraform fmt

The command formats Terraform files according to the standard Terraform formatting style. It does not create or modify cloud infrastructure.

19. Terraform Plan

The infrastructure changes were previewed using:

terraform plan

The plan shows that Terraform intends to create the aws_instance.web_server resource. Attributes that depend on AWS, such as the public IP, are shown as values known after apply.

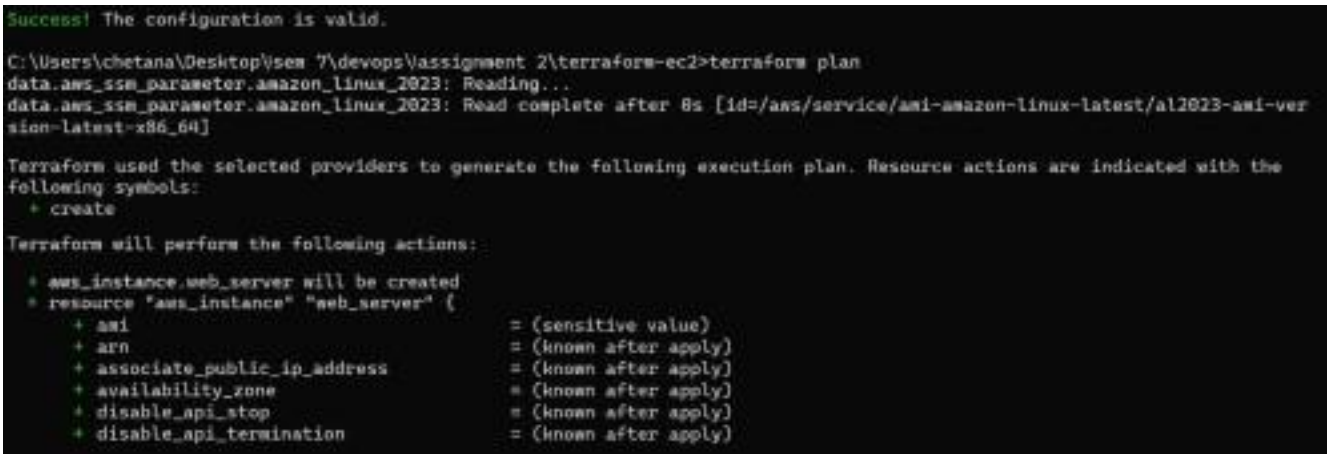


Figure 6: terraform plan – EC2 resource planned for creation

20. Terraform Apply

The configuration was deployed using:

terraform apply

After reviewing the proposed changes and confirming the operation, Terraform created the EC2 instance.

The supplied screenshot shows the apply operation and the resource creation process.

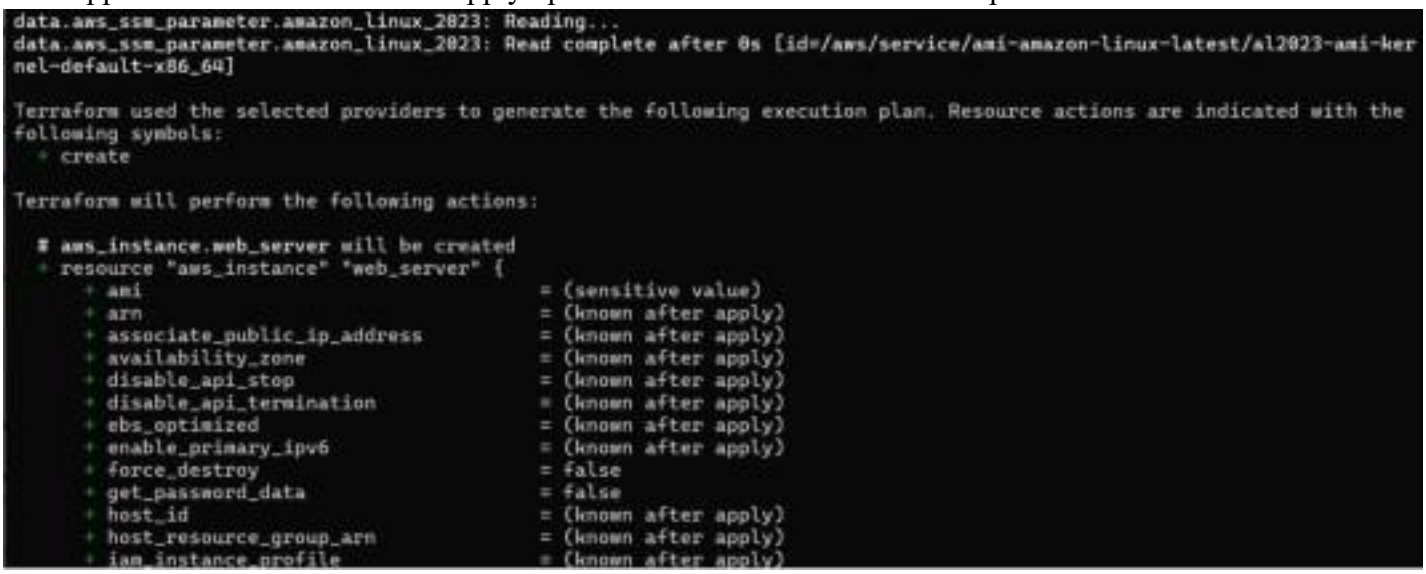


Figure 7: terraform apply – EC2 resource creation

21. Verify Terraform Outputs and State

Terraform state can be inspected using:

terraform show terraform state list

The supplied terraform show screenshot displays the AWS SSM parameter data source and the managed aws_instance.web_server resource. This confirms that Terraform has recorded the infrastructure in its state.

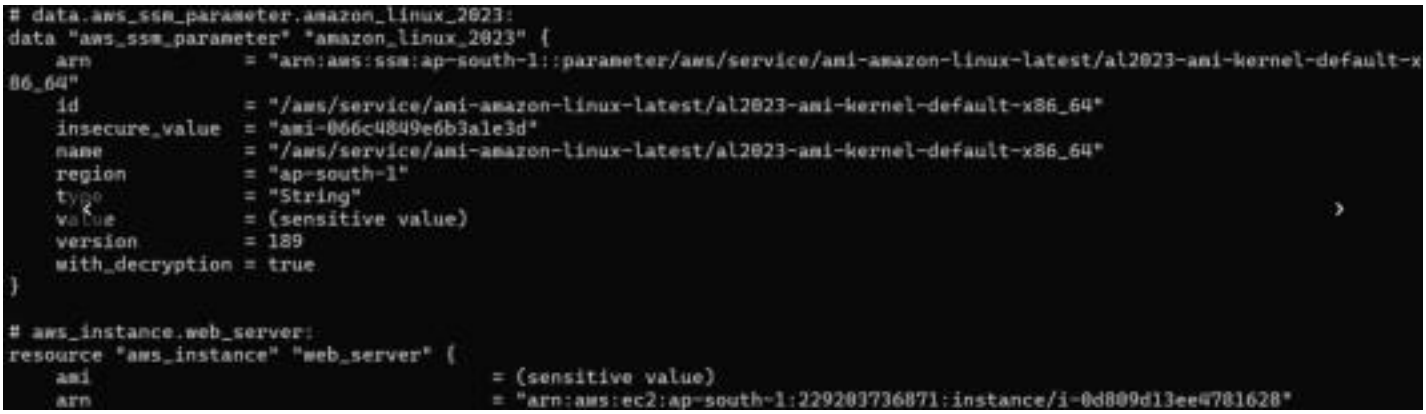


Figure 8: terraform show – Terraform state information The expected resource address in the state is:

aws_instance.web_server

22. Verify EC2 Instance in AWS

The EC2 console was checked after deployment. The supplied screenshot shows the instance named Terraform-EC2 in the Mumbai region, with instance type t3.micro and state Running.

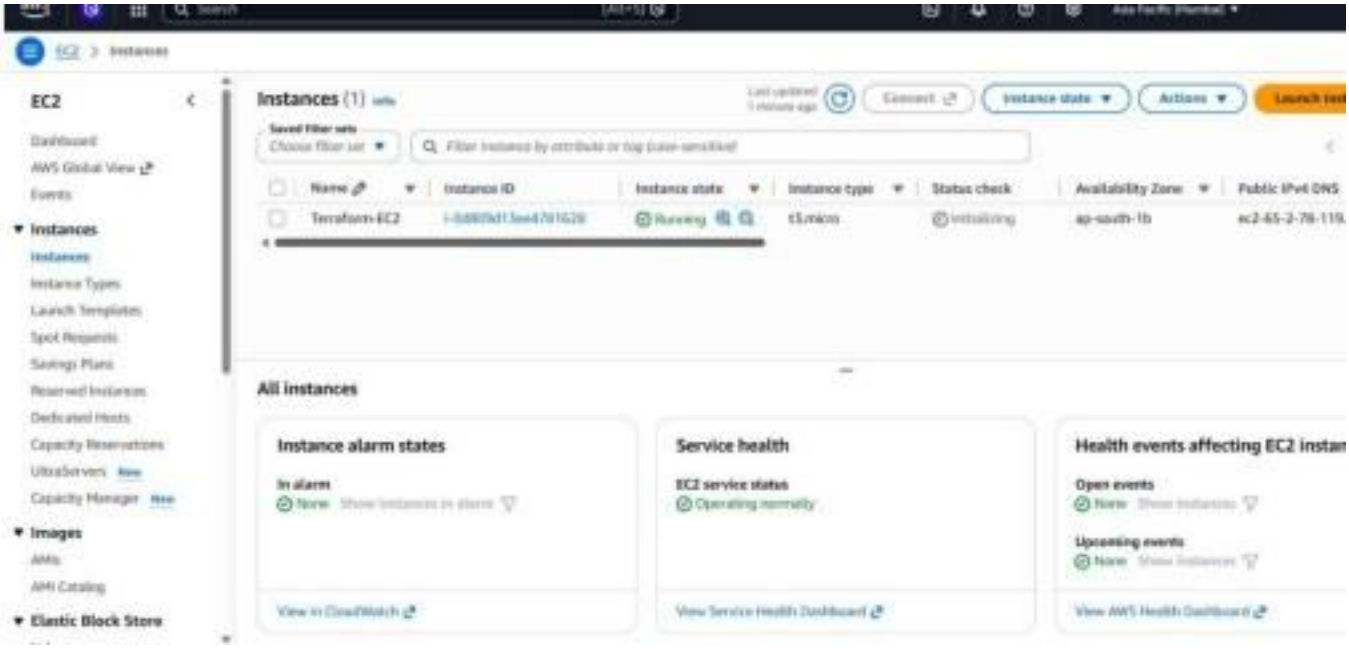


Figure 9: AWS EC2 Console – Terraform-EC2 instance running

23. Terraform Destroy

After completing the practical, the EC2 instance should be removed to avoid unnecessary cloud usage. The cleanup command is: terraform destroy

```
data.aws_ssm_parameter.amazon_linux_2023: Reading...
data.aws_ssm_parameter.amazon_linux_2023: Read complete after 0s [id=/aws/service/ami-amazon-linux-latest/al2023-ami-ker
nel-default-x86_64]
aws_instance.web_server: Refreshing state... [id=i-0d809d13ee4781628]

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the
following symbols:
  - destroy

Terraform will perform the following actions:

  # aws_instance.web_server will be destroyed
  - resource "aws_instance" "web_server" {
      - ami                                  = (sensitive value) -> null
      - arn                                  = "arn:aws:ec2:ap-south-1:229203736871:instance/i-0d809d13ee4781628" -> nul
l
      - associate_public_ip_address          = true -> null
      - availability_zone                    = "ap-south-1b" -> null
      - disable_api_stop                     = false -> null
      - disable_api_termination              = false -> null
      - ebs_optimized                        = false -> null
      - force_destroy                        = false -> null
```

24. Final Terraform Workflow

9. Create the Terraform configuration files.
10. Initialize the working directory using terraform init.
11. Validate the configuration using terraform validate.
12. Format the configuration using terraform fmt.
13. Preview infrastructure changes using terraform plan.
14. Create the EC2 instance using terraform apply.
15. Verify the instance in AWS and inspect Terraform state.
16. Remove the resource using terraform destroy.

25. Terraform File Architecture

File / Component	Purpose	Used in Practical

main.tf	AWS provider, AMI data source and EC2 resource	Yes
variables.tf	Input variables for instance type and name	Yes
outputs.tf	Displays EC2 information after deployment	Yes
.terraform.lock.hcl	Locks provider dependency selections	Generated by Terraform
terraform.tfstate	Tracksthe managed infrastructure state	Generated by Terraform

26. Difference Between Manual AWS and Terraform

Manual AWS Console	Terraform
Resources are created through GUI	Resources are created using code
Configuration is performed manually	Configuration is declared in .tf files
Difficult to reproduce exactly	Configuration can be reused
Changes depend heavily on manual actions	Changes can be planned before execution
Documentation may be separate	Configuration itself documents infrastructure
Resource deletion is manual	terraform destroy can remove managed resources
Versioning is difficult	Terraform files can be version controlled

27. Advantages of Terraform

17. Automation – reduces manual infrastructure configuration.
18. Repeatability – the same configuration can be reused.
19. Infrastructure Versioning – Terraform files can be stored in Git.
20. Planning – terraform plan allows changes to be reviewed before applying them. 21.
Resource Management – Terraform can create, modify and destroy managed resources.

22. Reusability – variables make configurations reusable across environments.
23. Consistency – infrastructure is created according to a defined configuration.

28. Limitations

· Cloud credentials must be configured securely.

· Incorrect configurations can create unwanted resources.

· Terraform state must be protected because it contains infrastructure information. · Cloud resources may generate charges if they are not removed.

· Provider versions and resource configurations must be maintained properly. **29.**

Observation Table

Parameter	Observation
IaC Tool	Terraform
Cloud Provider	AWS
Region	ap-south-1 (Mumbai)
Infrastructure Resource	EC2
Configuration Language	HCL
Provider	hashicorp/aws
Provider Version Observed	6.66.0 in the supplied terraform init screenshot
Resource	aws_instance.web_server
Instance Type	t3.micro
Instance Name	Terraform-EC2
AMI Selection	Amazon Linux 2023 SSM public parameter
Variables	Instance type, instance name
Outputs	Instance ID, public IP, public DNS
Initialization	terraform init

Validation	terraform validate
Formatting	terraform fmt
Planning	terraform plan
Deployment	terraform apply

State Inspection	terraform show / terraform state list
Cleanup	terraform destroy

30. Learning Outcomes

· Understanding of Infrastructure as Code.

· Understanding of Terraform architecture and workflow.

· Understanding of Terraform providers. ·

Understanding of Terraform resources.

· Understanding of Terraform variables.

· Understanding of Terraform outputs.

· Understanding of Terraform state.

· Provisioning an AWS EC2 instance through Terraform.

· Understanding infrastructure planning and deployment.

· Understanding automated infrastructure lifecycle management.

31. Conclusion

In this assignment, the concept of Infrastructure as Code (IaC) was studied using Terraform. Terraform wasinstalled and configured for AWS-based infrastructure provisioning. A Terraform configuration was developed using the AWS provider, an Amazon Linux 2023 AMI data source, an EC2 resource, input variables, and output values.

The Terraform workflow consisting of terraform init, terraform validate, terraform fmt, terraform plan, terraform apply, state inspection, and terraform destroy was studied as part of the practical. The supplied execution evidence demonstrates successful initialization, validation, planning, application, state inspection, and verification of the EC2 instance in the AWS console.

The practical demonstrates how Terraform can provision cloud infrastructure through code instead of relying only on manual AWS console configuration. It also demonstrates the benefits of automation, repeatability, consistency, planning, and infrastructure lifecycle management.

32. References

· HashiCorp Terraform Documentation.

· AWS Documentation – Amazon EC2.

· AWS Documentation – AWS Systems Manager Parameter Store. · Terraform
AWS Provider Documentation.

· Course material provided for Cloud Computing.